

Cloud Computing

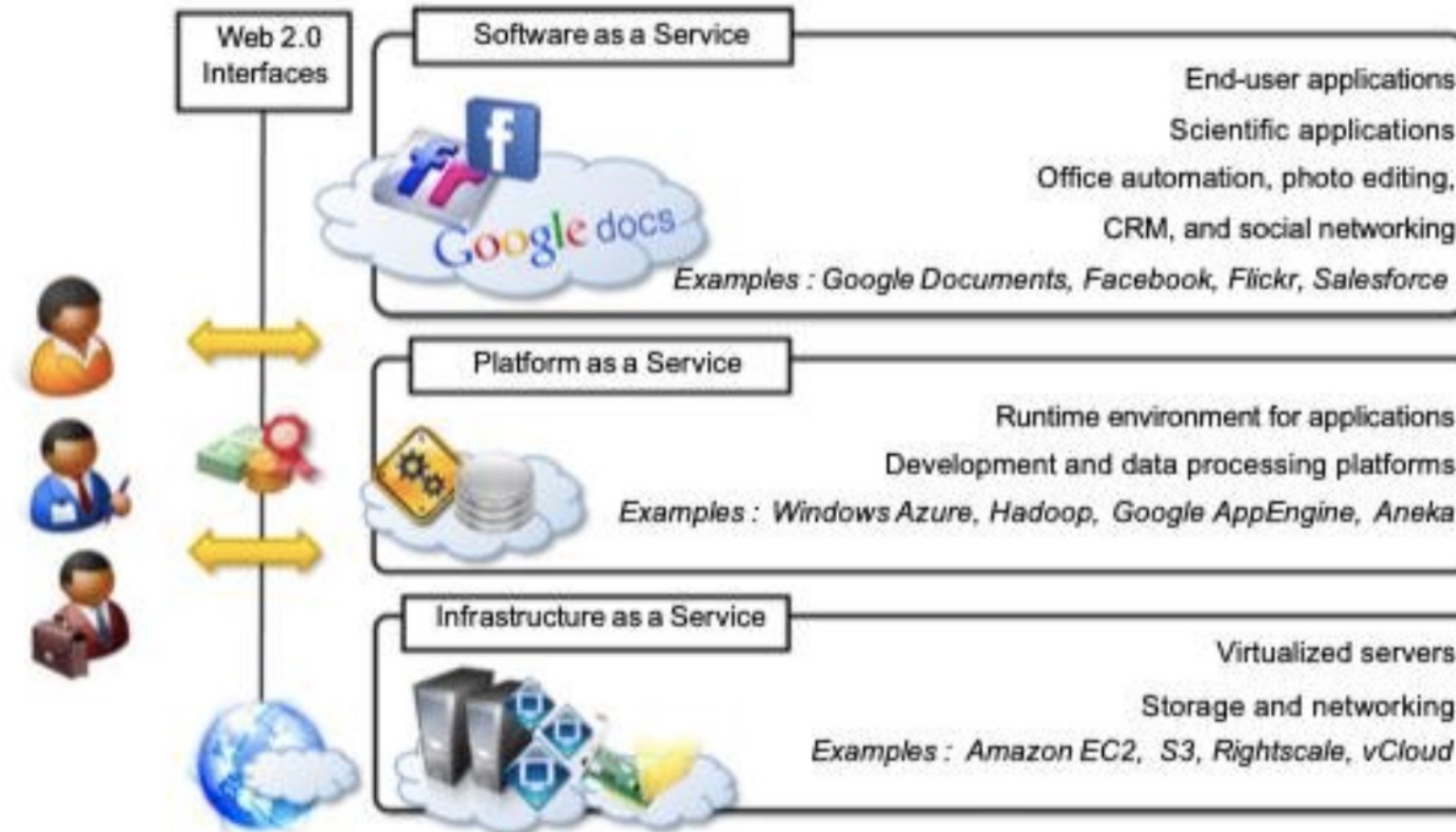
BCA 5th Semester, Batch BCA2024
School of Engineering

Presented by: Udaya Joisa

Cloud Computing Service Models

Fundamental characteristic of Cloud Computing is the capability of delivering a variety of IT services on demand.

Three major categories of Cloud Computing service offerings:



Cloud Computing Service Models

- **Infrastructure-as-a-Service (IaaS)**: At the base of the stack, IaaS solutions deliver infrastructure on demand in the form of virtual hardware, storage and networking. Provider's virtual hardware is utilised to provide compute on demand service in the form of virtual machine instances. Ex : Amazon EC2
- **Platform-as-a-Service(PaaS)**: PAAS solutions deliver scalable and elastic runtime environment on demand that enable building and hosting applications. These services are backed by a middleware platform that creates the abstract environment where applications are deployed and executed. The service provider provides scalability and manages fault-tolerance and user focuses on the application development by leveraging provider's APIs and libraries. Ex : Microsoft Azure App Service, AWS Elastic Beanstalk
- **Software-as-a-Service(SaaS)**: SaaS solutions provide applications and services on demand. Applications such as office automation, document management, photo editing, customer relationship management(CRM) software, Social Networking domains are made available on the provider's infrastructure, which are scalable and accessible through a browser on demand. Ex: Google Documents

Cloud Computing Service Models

- **IaaS** : VM with H/W with RAM, storage, network, OS
 - **PaaS**: Managed platforms for apps and databases.
-
- **IaaS** : You provision a VM with required storage, memory, OS image, network etc. and deploy rest of the software to develop you applications
 - **PaaS** : You deploy your application code on an abstract environment maintained by the CSP. CSP takes care of availability , scalability, maintenance etc.

Example :

- **IaaS** : Code and DB setup on a VM provisioned by you. Or Everything on your laptop (on-premisis IaaS)
 - **PaaS**: Code on VM and DB on AWS RDS.
-
- **AWS** : EC2 vs Elastic Beanstalk
 - **Azure**: Virtual Machines vs App Service.

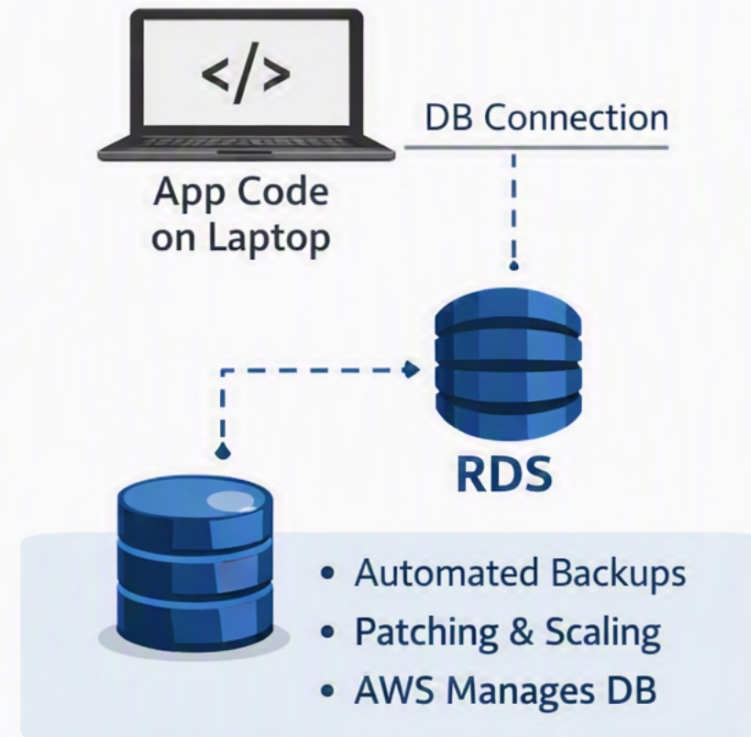
Cloud Computing Service Models

EC2 with Self-Managed Database



IaaS: FULL CONTROL OVER INFRASTRUCTURE

AWS RDS (Managed Database)



PaaS: MANAGED DATABASE SERVICE

IaaS and PaaS provisioning Workflow

IaaS Provisioning on AWS

Provisioning workflow:

1. Choose AMI (OS image).
2. Select instance type (compute/memory optimized).
3. Configure networking (VPC, subnets, security groups).
4. Attach storage (EBS, S3).
5. Launch and manage via **AWS Management Console** or **Terraform/CloudFormation**.

IaaS Provisioning on Azure

Provisioning workflow:

1. Select VM size and OS image.
2. Configure networking (VNets, NSGs, load balancers).
3. Attach disks (Premium SSD, Standard HDD).
4. Deploy via **Azure Portal**, **ARM templates**, or **Terraform**.

IaaS and PaaS provisioning Workflow

PaaS Provisioning on AWS

Provisioning workflow:

1. Upload application code.
2. Elastic Beanstalk provisions EC2, load balancers, and scaling automatically.
3. Configure environment variables and monitoring.
 - **Amazon RDS:** Managed relational databases (MySQL, PostgreSQL, Oracle, SQL Server).
 - **AWS Lambda** (serverless, but often grouped with PaaS): Event-driven execution without provisioning servers.

PaaS Provisioning on Azure

Provisioning workflow:

1. Deploy code via GitHub, Azure DevOps, or FTP.
2. App Service handles scaling, patching, and load balancing.
3. Configure monitoring with **Azure Monitor** and **Application Insights**.
 - **Azure SQL Database:** Fully managed relational database.
 - **Azure Functions:** Event-driven serverless compute.

The Vision of Cloud Computing

- The entire stack of a computing system is transformed into a **collection of utilities**, which can be provisioned and composed together to deploy systems in hours with virtually no maintenance costs.
- The vision of cloud computing is that the IT services are traded as utilities in an open market **without technological and legal barriers**.
- Create a global cloud **computing marketplace** by defining an effective standards for cloud services as well as the interaction among different cloud technologies.
- Shift towards cloud computing and its rapid adoption, facilitate **computing infrastructure consolidation**. Provide opportunities for optimising data centre facilities and fully utilising their capabilities to serve more users.
- **Reduce the waste of energy and carbon emission**, thus contributing to a greener IT on one hand and increase the revenue on the other end.

Cloud Terminologies - AWS

- Regions (minimum of two AZs)
- Availability zones (Cluster of Data centers)
- AWS instances, instance families – t2, m3, c4, r3, i2, d2 etc
- Fault tolerant
- Latency
- IOT
- 3-tier architecture
- ELB – Elastic Load Balancers
- Amazon EC2 – Elastic Compute Cloud - VM
- AMI – Amazon machine Image
- Compute environments – Sandbox, lab, dev, test, pre-prod, Prod, DR etc.
- AWS Marketplace - an online store to discover purchase and deploy IT software on top of the AWS infrastructure. Like AppStore, Playstore
- Cores Vs threads (physical components vs virtual constructs)
- i5 upto 12 cores, Xeon up to 128 cores.
 - Cores are processing units, threads are sequence of instructions
- KB vs KiB (KiBibyte), GB vs GiB (Gibibyte)

AWS Regions/AZs

**36 launched
Regions**

each with multiple
Availability Zones

**114 Availability
Zones**

**600+ CloudFront
POPs**

and 13 Regional edge caches

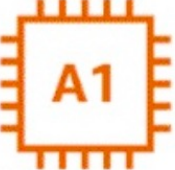
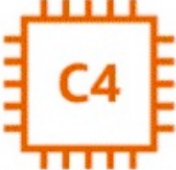
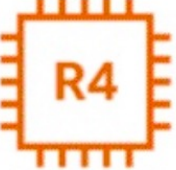
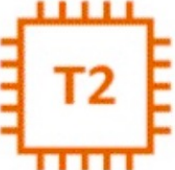
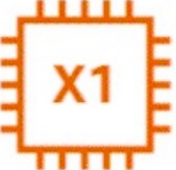
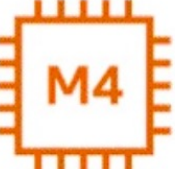
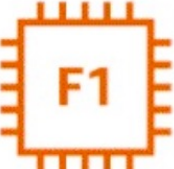
**42 Local Zones
29 Wavelength
Zones**

for ultralow-latency
applications

**245 countries and
territories served**

**135 Direct Connect
locations**

AWS Instances

General Purpose	Compute Optimised	Memory Optimised	Accelerated Computing	Storage Optimised
 A1 ARM based core and custom silicon	 C4 Compute - CPU intensive apps and DBs	 R4 RAM - Memory intensive apps and DB's	 P2 Processing optimised- Machine Learning	 H1 High Disk Throughput - Big data clusters
 T2 Tiny - Web servers and small DBs		 X1 Xtreme RAM - For SAP/Spark	 G3 Graphics Intensive - Video and streaming	 I3 IOPS - NoSQL DBs
 M4 Main - App servers and general purpose		 z1d High Compute and High Memory - Gaming	 F1 Field Programmable - Hardware acceleration	 D2 Dense Storage - Data Warehousing

Infrastructure as code (IaC)

Infrastructure as Code (IaC) is a method for setting up and managing computer systems using text files and code instead of manual clicking or physical setup. It enables teams build, change, configure, recreate and delete servers, networks, and databases quickly and safely.

Infrastructure as Code (IaC) is the method of managing and provisioning IT infrastructure using code rather than manual processes. It allows teams to define, deploy, and manage infrastructure through configuration files, which can be version-controlled and automated, similar to how application code is managed. This approach offers benefits like increased speed, consistency, and reliability in infrastructure management.

- Instead of manually configuring servers and other infrastructure through a GUI or command line, you define your infrastructure using code (e.g., using tools like *Terraform*, *AWS CloudFormation*, or *Azure Resource Manager templates*)

writie scripts that say:

- “I want a virtual server with 2 CPUs and 8 GB RAM.”
- “Connect this server to a database.”
- “Add security rules so only admins can access it.”

Game-changer/Benefits:

Consistency: Every environment (dev, staging, prod) gets built exactly the same way.

Version Control: Your infrastructure lives in a Git repo – just like your app code. Rollbacks and audits become easier.

Automation: Deploy infrastructure with CI/CD pipelines, just like you deploy applications.

Scalability: Create or delete resources at cloud speed, on demand.

Game-changer/benefits: *Continued.....*

- Stops human mistakes during setup.
- Makes copies of a system in minutes.
- Keeps test and live systems identical.
- Lets teams track who changed what

Infrastructure as code (IaC)

Popular Tools : Terraform, AWS CloudFormation, Ansible

Example (Terraform-style)

Hcl

 Copy

```
resource "aws_instance" "web_server" {  
  ami          = "ami-0123456789abcdefg"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "MyWebServer"  
  }  
}
```

That tiny block of code would provision a server on AWS. Imagine scaling that to an entire enterprise system – all in files you control.

Terraform

Terraform is an infrastructure as code (IaC) tool that allows you to build, change, and version infrastructure safely and efficiently. This includes low-level components such as compute instances, storage, and networking, as well as high-level components such as DNS entries, SaaS features, etc. Terraform can manage both existing service providers and custom in-house solutions.

- It is an open-source platform made for building, changing, and versioning infrastructure effectively and securely and is very straightforward.

Terraform

- Terraform utilizes HashiCorp Configuration Language (HCL) as its primary declarative language for defining infrastructure. HCL is designed to be human-readable and express infrastructure resources and their desired state.
- While HCL is the native language, Terraform also supports JSON as an alternative for configuration files. This allows users to define their infrastructure using a more widely recognized data serialization format if preferred.

Terraform Setup

1. Install Terraform
2. Install/configure AWS CLI
 - AWS Access Key ID:
 - AWS Secret Access Key:
 - Default region name:
 - Default output format:
3. Create [main.tf](#)

Terraform Setup

main.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-company-demo-object-storage-2026"

  tags = {
    Name          = "My Object Storage"
    Environment   = "Dev"
  }
}
```

Terraform Setup

Commands to run

- terraform init
- terraform validate
- terraform plan
- terraform apply
- terraform destroy

Terraform Example

Create a bucket and upload a file

```
provider "aws" {  
  region = "ap-south-1"    # Change region as needed  
}  
  
# Create S3 bucket  
resource "aws_s3_bucket" "my_bucket" {  
  bucket = "uj-cu-bucket"    # Change bucket name (must be globally unique)  
}  
  
# Upload a PDF file to the bucket  
resource "aws_s3_object" "pdf_upload" {  
  bucket = aws_s3_bucket.my_bucket.bucket  
  key    = "uj1.pdf"          # Object name in S3  
  source = "uj1.pdf"          # Local path to your PDF file  
}
```

Terraform Example

Make the uploaded file public

```
resource "aws_s3_bucket" "my_bucket" {  
    bucket = "u1-public-pdf-bucket"  
    acl    = "private"  
    # Disable block public access  
    block_public_acls    = false  
    block_public_policy  = false  
    ignore_public_acls   = false  
    restrict_public_buckets = false  
}  
  
resource "aws_s3_bucket_object" "pdf_upload" {  
    bucket = aws_s3_bucket.my_bucket.bucket  
    key    = "sample.pdf"  
    source = "files/sample.pdf"  
    acl    = "public-read" # This makes the object readable by anyone  
}
```

<https://<bucket-name>.s3.amazonaws.com/sample.pdf>

Terraform Example 2 – With policy (better approach)

```
provider "aws" {  
    region = "us-east-1"  
}  
  
# Create S3 bucket  
resource "aws_s3_bucket" "my_bucket" {  
    bucket = "u1-public-pdf-bucket" # Must be globally unique  
    acl    = "private"  
}  
  
# Upload PDF file  
resource "aws_s3_bucket_object" "pdf_upload" {  
    bucket = aws_s3_bucket.my_bucket.bucket  
    key    = "sample.pdf"  
    source = "files/sample.pdf"  
    acl    = "private" # Keep object private, policy will override  
}
```

Terraform Example

```
# Public read bucket policy

resource "aws_s3_bucket_policy" "public_read" {

  bucket = aws_s3_bucket.my_bucket.id

  policy = <<POLICY
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::u1-public-pdf-bucket/*"
    }
  ]
}
POLICY
}
```

laC tools

- **Terraform / OpenTofu**: Builds servers and networks.
- **Ansible / Chef**: Sets up software on those servers.
- **AWS CloudFormation**: Manages setups inside Amazon Cloud